

Advanced Python: Functions, Classes and PyTorch nn.Module

Module 2 of 4 — Python and Machine Learning with PyTorch

Sayan CHAKI

Chercheur
LIRIS, Ecole Centrale Lyon, Univ. Lyon 2, INSA

September 9, 2026

Table of Contents

- 1 Functions in Depth
- 2 Object-Oriented Python
- 3 From Classes to `nn.Module`
- 4 Practical Session

Recap of Module 1 and today's goal

You already know: tensors, shapes, devices, elementwise vs matrix products.

Today's single big idea

A neural network in PyTorch is a **Python class**. If you understand classes, you understand `nn.Module`. There is no extra magic.

Path of the lecture: functions $\rightarrow$ classes $\rightarrow$ `nn.Module` $\rightarrow$ a real training loop.

Where we are

- 1 Functions in Depth
- 2 Object-Oriented Python
- 3 From Classes to `nn.Module`
- 4 Practical Session

Scope: local, enclosing, global

```
lr = 0.01                # global

def train():
    lr = 0.1              # LOCAL: shadows the global, does not change it
    print(lr)             # 0.1

train()
print(lr)                 # 0.01 -> the global is untouched

def counter():
    count = 0
    def increment():      # enclosing scope
        nonlocal count    # without this, count would be local to increment
        count += 1
        return count
    return increment
```

LEGB rule

Python resolves a name in this order: **L**ocal, **E**nclosing, **G**lobal, **B**uilt-in.

The mutable default argument trap

```
# WRONG: the list is created ONCE, at function definition time
def add_loss(value, history=[]):
    history.append(value)
    return history

add_loss(1.0)    # [1.0]
add_loss(2.0)    # [1.0, 2.0]    <- surprise, the list persisted!

# CORRECT
def add_loss(value, history=None):
    if history is None:
        history = []
    history.append(value)
    return history
```

Remember

Never use a mutable object (list, dict, tensor) as a default argument.

Lambdas and higher-order functions

```
square = lambda x: x ** 2      # anonymous, single-expression function
square(4)                      # 16

pairs = [("resnet", 0.91), ("vgg", 0.87), ("mlp", 0.72)]
pairs.sort(key=lambda p: p[1], reverse=True)  # sort by accuracy

accs = list(map(lambda p: p[1], pairs))        # map
good = list(filter(lambda p: p[1] > 0.8, pairs)) # filter

# In PyTorch you will meet lambdas inside transforms:
# transforms.Lambda(lambda x: x.flatten())
```

When to use a lambda

Only for very short throwaway logic, typically as a `key=` argument. Anything longer deserves a named `def`.

Generators: lazy sequences

```
def batch_indices(n, batch_size):  
    """Yields index chunks WITHOUT building the whole list in memory."""  
    for start in range(0, n, batch_size):  
        yield list(range(start, min(start + batch_size, n)))  
  
for idx in batch_indices(10, 4):  
    print(idx)          # [0,1,2,3] then [4,5,6,7] then [8,9]  
  
# Generator expression: like a list comprehension with ( ) instead of [ ]  
total = sum(x ** 2 for x in range(1_000_000))  # constant memory
```

Why this matters

A PyTorch DataLoader is an iterable that yields one batch at a time. That is exactly the generator idea, applied to datasets too large for RAM.

Where we are

- 1 Functions in Depth
- 2 Object-Oriented Python
- 3 From Classes to `nn.Module`
- 4 Practical Session

A first class

```
class Dataset:
    """A blueprint. Nothing exists until you instantiate it."""

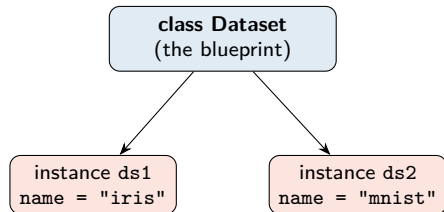
    def __init__(self, name, samples):    # the CONSTRUCTOR
        self.name = name                 # attribute stored on the instance
        self.samples = samples

    def __len__(self):                    # enables len(obj)
        return len(self.samples)

    def mean(self):                       # a METHOD
        return sum(self.samples) / len(self.samples)

ds = Dataset("iris", [5.1, 4.9, 4.7])    # __init__ runs here
print(ds.name, len(ds), ds.mean())      # iris 3 4.9
```

Class vs instance, and what self means



- The **class** is written once.
- Each **instance** has its own attributes.
- `self` is the instance currently being worked on. Python passes it automatically: `ds.mean()` really calls `Dataset.mean(ds)`.
- `__init__` is called automatically at construction. It does not return anything.

Inheritance and super()

```
class BaseModel:
    def __init__(self, name):
        self.name = name
    def describe(self):
        return f"Model {self.name}"

class Classifier(BaseModel):           # inherits from BaseModel
    def __init__(self, name, n_classes):
        super().__init__(name)        # run the PARENT constructor
        self.n_classes = n_classes    # then add our own attributes

    def describe(self):                # OVERRIDE the parent method
        return super().describe() + f" with {self.n_classes} classes"

clf = Classifier("mlp", 10)
print(clf.describe())                 # Model mlp with 10 classes
```

Golden rule for PyTorch

Forgetting `super().__init__()` in an `nn.Module` subclass breaks parameter registration. It is the number one error in this course.

Dunder methods make objects feel native

```
class Sample:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __repr__(self):
        # what print() shows
        return f"Sample(x={self.x}, y={self.y})"

    def __len__(self):
        # enables len(s)
        return len(self.x)

    def __getitem__(self, i):
        # enables s[i] and iteration
        return self.x[i]

s = Sample([1, 2, 3], y=0)
print(s)           # Sample(x=[1, 2, 3], y=0)
print(len(s))      # 3
print(s[1])        # 2
```

Preview of Module 3

`__len__` and `__getitem__` are exactly the two methods a custom PyTorch Dataset must implement.

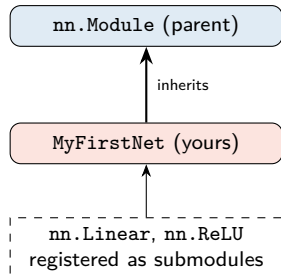
Where we are

- 1 Functions in Depth
- 2 Object-Oriented Python
- 3 From Classes to `nn.Module`**
- 4 Practical Session

What `nn.Module` gives you for free

By subclassing `torch.nn.Module` you inherit:

- `.parameters()`: every learnable tensor, collected automatically.
- `.to(device)`: moves the whole model in one call.
- `.train()` / `.eval()`: switches dropout and batch norm.
- `.state_dict()`: save and load weights.
- `.zero_grad()`: reset gradients.
- Calling the object runs `forward()` plus the hook machinery.



The canonical model skeleton

```
import torch
import torch.nn as nn

class MyFirstNet(nn.Module):
    def __init__(self, in_features=4, hidden=16, n_classes=3):
        super().__init__()          # MANDATORY first line
        self.fc1 = nn.Linear(in_features, hidden)  # layer DEFINITION
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden, n_classes)

    def forward(self, x):             # layer USE
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x                     # raw scores, called logits

model = MyFirstNet()
dummy = torch.randn(8, 4)          # a batch of 8 samples, 4 features each
out = model(dummy)                  # calls forward() under the hood
print(out.shape)                    # torch.Size([8, 3])
```


`__init__` versus `forward`: the key distinction

<code>__init__(self, ...)</code>	<code>forward(self, x)</code>
Runs once , at construction.	Runs every time data passes through.
Declares <i>what the model owns</i> .	Declares <i>what the model does</i> .
Creates layers and allocates weights.	Applies layers to the input tensor.
No data involved.	The batch <code>x</code> flows through here.
Registers parameters with the parent.	Builds the autograd graph.

Never call `model.forward(x)` directly

Write `model(x)`. The `__call__` wrapper also runs registered hooks, which some PyTorch features rely on.

Inspecting a model

```
model = MyFirstNet(in_features=4, hidden=16, n_classes=3)

print(model)
# MyFirstNet(
#   (fc1): Linear(in_features=4, out_features=16, bias=True)
#   (relu): ReLU()
#   (fc2): Linear(in_features=16, out_features=3, bias=True)
# )

for name, p in model.named_parameters():
    print(name, tuple(p.shape), p.requires_grad)
# fc1.weight (16, 4) True
# fc1.bias   (16,)  True
# fc2.weight (3, 16) True
# fc2.bias   (3,)   True

n_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print("trainable parameters:", n_params)      # 16*4+16 + 3*16+3 = 131
```

nn.Sequential and nn.ModuleList

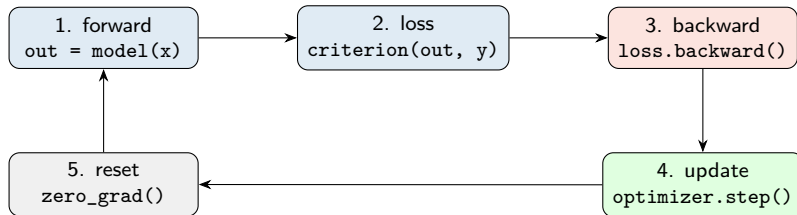
```
# Shortcut when the model is a plain stack, no branching
mlp = nn.Sequential(
    nn.Linear(4, 16),
    nn.ReLU(),
    nn.Linear(16, 3),
)

# Inside a class, for a variable number of layers
class DeepNet(nn.Module):
    def __init__(self, sizes):
        super().__init__()
        self.layers = nn.ModuleList(
            [nn.Linear(a, b) for a, b in zip(sizes[:-1], sizes[1:])]
        )
    def forward(self, x):
        for i, layer in enumerate(self.layers):
            x = layer(x)
            if i < len(self.layers) - 1:
                x = torch.relu(x)
        return x
```

Do not use a plain Python list

A normal [] list of layers is invisible to `.parameters()`. Use `nn.ModuleList` or `nn.Sequential`.

The training loop, seen as a cycle



Five lines, repeated for every batch of every epoch. Every deep learning script you will ever read contains this cycle.

A complete, runnable training loop

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = MyFirstNet(4, 16, 3).to(device)

criterion = nn.CrossEntropyLoss()          # expects raw logits + int labels
optimizer = torch.optim.Adam(model.parameters(), lr=1e-2)

X = torch.randn(64, 4, device=device)
y = torch.randint(0, 3, (64,), device=device)

model.train()
for epoch in range(20):
    optimizer.zero_grad()                  # 5. clear old gradients
    logits = model(X)                      # 1. forward
    loss = criterion(logits, y)             # 2. loss
    loss.backward()                         # 3. backward
    optimizer.step()                       # 4. update
    if epoch % 5 == 0:
        print(f"epoch {epoch:3d} | loss {loss.item():.4f}")
```

Forgetting `zero_grad()`

Gradients **accumulate** by default. Without the reset, your updates are wrong and training silently diverges.

Evaluation mode and no_grad

```
model.eval()                                # dropout off, batch norm uses running stats
with torch.no_grad():                       # no graph built -> faster, less memory
    logits = model(X_test)
    preds = logits.argmax(dim=1)            # index of the largest score per row
    acc = (preds == y_test).float().mean()
print(f"accuracy = {acc.item():.3f}")

model.train()                               # switch back before training again
```

Two independent switches

- `eval()` / `train()` changes **layer behaviour**.
- `no_grad()` changes **gradient tracking**.

At test time you almost always want both.

Saving and loading

```
# Recommended: save only the weights, not the Python object
torch.save(model.state_dict(), "myfirstnet.pt")

# Later, in a fresh session:
model = MyFirstNet(4, 16, 3) # rebuild the SAME architecture
model.load_state_dict(torch.load("myfirstnet.pt", map_location="cpu"))
model.eval()
```

Why not `torch.save(model)`?

Pickling the whole object hard-codes the class path. If you rename or move your file, loading breaks. The `state_dict` is just a dictionary of tensors and is portable.

Where we are

- 1 Functions in Depth
- 2 Object-Oriented Python
- 3 From Classes to `nn.Module`
- 4 Practical Session

Colab Notebook 2: Classes & `nn.Module`

What you will do:

- Write functions with `*args`, lambdas and a generator that yields batches.
- Build a plain Python class, then a subclass with `super()`.
- Define `MyFirstNet(nn.Module)` with `nn.Linear` layers in `__init__` and the logic in `forward`.
- Push a dummy tensor through it and read the output shape.
- Inspect `named_parameters()` and count trainable weights.
- Run a full training loop on synthetic data and watch the loss fall.
- **Challenge:** make the network depth a constructor argument using `nn.ModuleList`.

File: `Colab_Notebook_2_Classes.ipynb`

Key takeaways

- 1 Functions have scope; never use mutable defaults.
- 2 Generators produce values lazily, which is how DataLoader works.
- 3 A class bundles data (attributes) and behaviour (methods); `self` is the current instance.
- 4 `nn.Module` is a normal parent class. `super().__init__()` is mandatory.
- 5 `__init__` declares layers, `forward` uses them. Call `model(x)`, never `model.forward(x)`.
- 6 The training loop is always the same five steps.

Next module

Feeding real data into this loop with Dataset and DataLoader.

Questions?

See you in Module 3.